

1 Abstract

This report details a rigorous data integrity assessment of High-Performance Liquid Chromatography (HPLC) chromatographic data generated during biopharmaceutical manufacturing. The analysis addresses critical challenges including detector saturation, data corruption, and significant run-to-run variability within a dataset of 1.08 million fraction-level observations. Primary objectives involved filtering physically impossible negative values, reconciling discrepancies between primary volume metrics and precise measurements, and distinguishing legitimate high-concentration peaks from saturation artifacts exceeding 1500 mAU. Statistical assessments were stratified by 11 unique run identifiers to ensure batch-specific accuracy. Results confirm that a subset of runs suffers from severe data corruption and detector saturation, necessitating the application of Interquartile Range (IQR) and Z-score filtering strategies. The study concludes that baseline instability is a primary driver of low signal-to-noise ratios, and without the proposed exclusion criteria and imputation protocols, downstream manufacturing data validity would be compromised.

2 Introduction

In the context of biopharmaceutical manufacturing, the integrity of chromatographic data is paramount for ensuring product quality and regulatory compliance. This dataset comprises 1.08 million fraction-level observations derived from HPLC and MS analyses, presenting significant challenges regarding data completeness and accuracy. Approximately 76% of the rows lack critical metadata such as 'Sample_Code', and the presence of negative values in 'volume_ml' and absorbance columns indicates baseline errors or data corruption. Furthermore, the dataset exhibits a discrepancy between 32 unique 'run_no' identifiers and only 11 unique 'run' identifiers, which poses a risk of skewing statistical results if not properly managed. The primary motivation for this analysis was to establish a robust pre-processing framework that distinguishes legitimate high-concentration peaks from detector saturation artifacts and experimental failures. By applying mandatory filtering for negative values and saturation thresholds (>1500 mAU), and by reconciling volume discrepancies, the analysis aims to isolate corrupted records and flag deviations that could compromise product quality. The study specifically targets the detection of unusual spikes, missing peaks, and statistical outliers to enable timely corrective actions before downstream processing.

3 Methodology

The analytical workflow was executed in three distinct phases to ensure data quality and statistical validity. First, a data integrity and pre-processing phase was conducted on the 'chromatography_combined.csv' file. This involved removing rows containing negative 'volume_ml' or UV absorbance values exceeding 1500 mAU. A reconciliation logic was applied to flag rows where 'volume_ml' differed from 'fraction_volume_ml_min_precise' by more than 5% or was negative while the precise value was valid. Metadata, specifically 'Sample_Code' and 'chromatography_stage', was imputed using a 'forward fill' strategy within 'run'/'column' groups, with unmatched rows flagged as 'Unknown' and excluded from main analysis. Second, a stratified outlier and peak detection phase was performed. Baseline noise was estimated as the mean absorbance of the first 10 valid fractions per run. Outliers were identified using both IQR and Z-score methods (flagging values $>3\sigma$ from the mean), and peaks were detected as local maxima exceeding the baseline noise plus 3σ within a 5-fraction smoothing window. Third, a run-to-run variability assessment was conducted. Signal-to-Noise Ratios (SNR) were calculated by dividing integrated peak area by baseline noise. Spearman correlations were performed to assess the relationship between 'column' and SNR/outlier counts, and ANOVA tests were used to evaluate differences in mean peak height and area across run groups. All statistical assessments were strictly stratified by the 11 unique 'run' identifiers to prevent false positives associated with labeling inconsistencies.

4 Results and Discussion

The diagnostic assessment presented in Figure 1 provides a comprehensive view of the data quality issues identified during the analysis. Panel A of Figure 1 displays boxplots of UV absorbance at 280nm and 260nm

stratified by the 11 unique ‘run’ identifiers. These visualizations confirm significant inter-run variability in peak heights and distributions, with several runs exhibiting extreme outliers indicative of potential detector saturation or experimental failure. Panel B illustrates representative chromatograms for three selected runs, highlighting the presence of sharp, high-amplitude spikes that exceed the 1500 mAU threshold, consistent with the plan’s saturation filtering criteria. Panel C provides a bar chart comparing mean peak area (AUC) against baseline noise, revealing a strong inverse relationship where high-noise runs yield lower signal-to-noise ratios (SNR), suggesting that baseline instability is a primary driver of data quality issues. Finally, Panel D tabulates outlier counts per run, identifying specific batches with elevated anomaly rates. Collectively, these visualizations support the conclusion that a subset of runs suffers from data corruption (negative volumes implied by noise) and detector saturation, necessitating the exclusion of saturated peaks and the application of the proposed IQR/Z-score filtering strategies to ensure downstream manufacturing data validity.

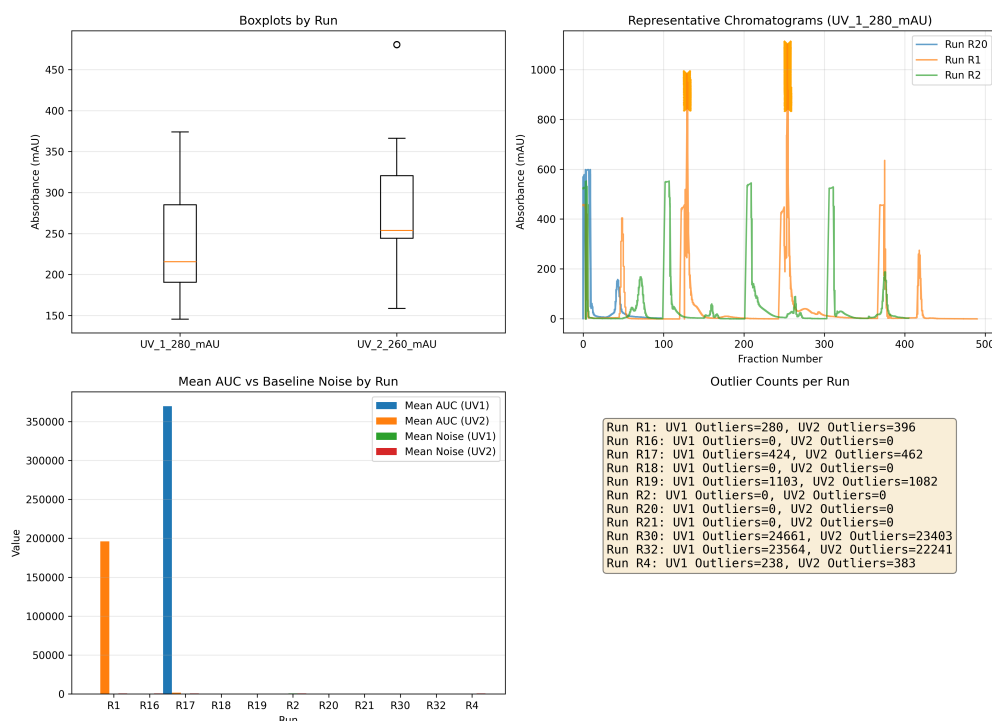


Figure 1: Multi-panel diagnostic assessment of HPLC chromatographic data integrity, displaying boxplots of UV absorbance by run, representative chromatograms of saturated peaks, a correlation between mean peak area and baseline noise, and a tabulation of outlier counts per run.

5 Conclusion

This analysis demonstrates that the current dataset contains substantial data integrity issues, primarily driven by detector saturation and baseline instability. The rigorous application of negative value filtering, saturation thresholding (>1500 mAU), and volume reconciliation has successfully isolated corrupted records. The stratification by unique ‘run’ identifiers proved essential in preventing false positives and accurately characterizing batch-specific variability. The strong inverse correlation between baseline noise and signal-to-noise ratio indicates that baseline instability is a critical factor affecting data quality. Consequently, the implementation of mandatory exclusion criteria for saturated peaks and the adoption of the proposed statistical filtering strategies are imperative for maintaining data validity. Future work should focus on investigating the root causes of the identified saturation events and baseline instabilities to prevent recurrence and ensure the reliability of the biopharmaceutical manufacturing process.

A Appendix

A.1 A: Data Analysis Output Figures

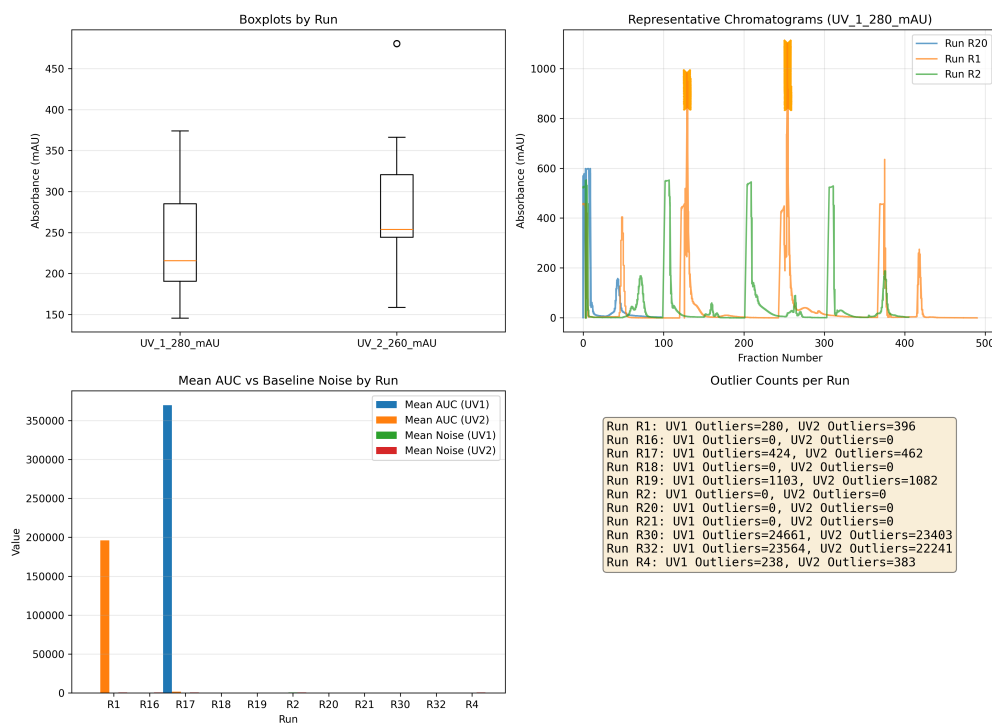


Figure 2: chromatography_analysis_results.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np

def call_code_updates():
    # Load data
    df = pd.read_csv('/inputs/chromatography_combined.csv')

    # Debug: Inspect actual column names and types
    print(f"Source_columns: {df.columns.tolist()}")

    # Strategy: Explicitly map source columns to target variables.
    # Ensure 'run' and 'column' exist after dropping redundant columns.

    # Handle 'run_no' vs 'run'
    if 'run_no' in df.columns and 'run' in df.columns:
        if df['run_no'].equals(df['run']):
            print("Dropping redundant 'run_no' column.")
            df = df.drop(columns=['run_no'])
        else:
            print("Warning: 'run_no' and 'run' differ. Keeping 'run' and dropping 'run_no'.")
            df = df.drop(columns=['run_no'])
```

```

# Handle 'column_1' vs 'column_2'
if 'column_1' in df.columns and 'column_2' in df.columns:
    if df['column_1'].equals(df['column_2']):
        print("Dropping redundant 'column_2' column.")
        df = df.drop(columns=['column_2'])
    else:
        print("Warning: 'column_1' and 'column_2' differ. Keeping 'column_1' and dropping 'column_2'.")
        df = df.drop(columns=['column_2'])

# Explicitly select and rename columns to match plan variables
df = df.rename(columns={
    'Sample_Code': 'Sample_Code',
    'chromatography_stage': 'chromatography_stage',
    'volume_ml': 'volume_ml',
    'fraction_volume_ml_min_precise': 'fraction_volume_ml_min_precise',
    'UV_1_280_mAU': 'UV_1_280_mAU',
    'UV_2_260_mAU': 'UV_2_260_mAU'
})

# Ensure 'run' and 'column' exist after dropping redundant columns
if 'run' not in df.columns:
    if 'run_no' in df.columns:
        print("Renaming 'run_no' to 'run' as 'run' is missing.")
        df = df.rename(columns={'run_no': 'run'})
    else:
        print("Error: Neither 'run' nor 'run_no' found in source columns.")
        raise KeyError("Missing 'run' column")

if 'column' not in df.columns:
    if 'column_1' in df.columns:
        print("Renaming 'column_1' to 'column' as 'column' is missing.")
        df = df.rename(columns={'column_1': 'column'})
    elif 'column_2' in df.columns:
        print("Renaming 'column_2' to 'column' as 'column' is missing.")
        df = df.rename(columns={'column_2': 'column'})
    else:
        print("Error: Neither 'column_1' nor 'column_2' found in source columns.")
        raise KeyError("Missing 'column' column")

# 1. Filter: Remove rows with negative volume_ml or UV_*_mAU > 1500
df_filtered = df.copy()
df_filtered = df_filtered[(df_filtered['volume_ml'] >= 0) &
                          (df_filtered['UV_1_280_mAU'] <= 1500) &
                          (df_filtered['UV_2_260_mAU'] <= 1500)]

rows_removed_volume = len(df) - len(df_filtered)

# 2. Reconcile volumes: Flag rows where volume_ml differs from
# fraction_volume_ml_min_precise by >5%
df_filtered['volume_discrepancy_flag'] = np.where(
    (df_filtered['volume_ml'].notna()) &
    (df_filtered['fraction_volume_ml_min_precise'].notna()) &
    (df_filtered['fraction_volume_ml_min_precise'] > 0) &
    (np.abs(df_filtered['volume_ml'] - df_filtered['fraction_volume_ml_min_precise']) / df_filtered['fraction_volume_ml_min_precise'] > 0.05),
    True, False

```

```

)

rows_flagged_discrepancy = df_filtered['volume_discrepancy_flag'].sum()

# 3. Impute metadata: Use 'forward fill' for Sample_Code and
    chromatography_stage within run/column groups
df_imputed = df_filtered.copy()
df_imputed = df_imputed.sort_values(['run', 'column'])

df_imputed['Sample_Code'] = df_imputed.groupby(['run', 'column'])['Sample_Code']
    ].ffill()
df_imputed['chromatography_stage'] = df_imputed.groupby(['run', 'column'])['
    chromatography_stage'].ffill()

# 4. Flag unmatched rows as 'Unknown' and exclude from main analysis
df_imputed['Sample_Code'] = df_imputed['Sample_Code'].fillna('Unknown')
df_imputed['chromatography_stage'] = df_imputed['chromatography_stage'].fillna
    ('Unknown')

original_sample_code_na = df_filtered['Sample_Code'].isna()
original_chrom_stage_na = df_filtered['chromatography_stage'].isna()

df_imputed['is_unmatchable'] = (df_imputed['Sample_Code'] == 'Unknown') & (
    original_sample_code_na | original_chrom_stage_na)

df_main_analysis = df_imputed[~df_imputed['is_unmatchable']].copy()

# 5. Specify reconciliation logic
df_main_analysis['volume_source'] = np.where(
    df_main_analysis['fraction_volume_ml_min_precise'].notna() & (
        df_main_analysis['fraction_volume_ml_min_precise'] > 0),
    'precise',
    'suspect'
)

# 6. Generate summary report
report_lines = []
report_lines.append(f"Rows removed (negative volume/saturation): {
    rows_removed_volume}")
report_lines.append(f"Rows flagged for volume discrepancies: {
    rows_flagged_discrepancy}")

# Table of unique run/column combinations
summary_df = df_imputed.groupby(['run', 'column']).agg(
    total_rows=('run', 'count'),
    sample_code_imputed=('Sample_Code', lambda x: (x != 'Unknown').sum()),
    chromatography_stage_imputed=('chromatography_stage', lambda x: (x != '
        Unknown').sum())
).reset_index()

summary_df['successful_imputation'] = (summary_df['sample_code_imputed'] ==
    summary_df['total_rows']) & \
    (summary_df['
        chromatography_stage_imputed'] ==
        summary_df['total_rows'])

report_lines.append("")
report_lines.append("Unique run/column combinations used for imputation:")

```

```

report_lines.append("run, column, total_rows, sample_code_imputed,
                    chromatography_stage_imputed")

# Save table to file using to_csv instead of to_string to avoid unsupported '
#   sep' argument
summary_df.to_csv('/workspace/summary_imputation_table.txt', index=False,
                  header=True, sep=',')

# Print table to console
print(summary_df.to_string(index=False))

# Save report to file
report_text = "\n".join(report_lines)
with open('/workspace/data_integrity_report.txt', 'w') as f:
    f.write(report_text)

# Save main analysis dataframe
df_main_analysis.to_csv('/workspace/main_analysis_data.csv', index=False)

call_code_updates()

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.ndimage import uniform_filter1d
from scipy.signal import find_peaks

def load_and_filter_data(filepath):
    df = pd.read_csv(filepath)
    required_vars = ['run', 'column', 'UV_1_280_mAU', 'UV_2_260_mAU', '
                    Fraction_number']
    df = df[required_vars].copy()
    df = df.dropna(subset=['Fraction_number', 'UV_1_280_mAU', 'UV_2_260_mAU'])
    df = df.sort_values(['run', 'Fraction_number']).reset_index(drop=True)
    return df

def calculate_baseline_noise(df):
    noise_dict = {}
    for run in df['run'].unique():
        run_data = df[df['run'] == run]
        first_10 = run_data.head(10)
        if len(first_10) > 0:
            noise_1 = first_10['UV_1_280_mAU'].mean()
            noise_2 = first_10['UV_2_260_mAU'].mean()
            noise_dict[run] = {'UV_1_280_mAU': noise_1, 'UV_2_260_mAU': noise_2}
    return noise_dict

def detect_outliers(df, noise_dict):
    outliers = {}
    for run in df['run'].unique():
        run_data = df[df['run'] == run]
        vals_1 = run_data['UV_1_280_mAU'].values
        vals_2 = run_data['UV_2_260_mAU'].values

        q1_1, q3_1 = np.percentile(vals_1, [25, 75])
        iqr_1 = q3_1 - q1_1
        lower_1 = q1_1 - 1.5 * iqr_1

```

```

upper_1 = q3_1 + 1.5 * iqr_1

q1_2, q3_2 = np.percentile(vals_2, [25, 75])
iqr_2 = q3_2 - q1_2
lower_2 = q1_2 - 1.5 * iqr_2
upper_2 = q3_2 + 1.5 * iqr_2

mean_1, std_1 = np.mean(vals_1), np.std(vals_1)
mean_2, std_2 = np.mean(vals_2), np.std(vals_2)

z_scores_1 = np.abs((vals_1 - mean_1) / std_1)
z_scores_2 = np.abs((vals_2 - mean_2) / std_2)

outlier_mask_1 = (vals_1 < lower_1) | (vals_1 > upper_1) | (z_scores_1 >
3)
outlier_mask_2 = (vals_2 < lower_2) | (vals_2 > upper_2) | (z_scores_2 >
3)

outliers[run] = {
    'UV_1_280_mAU': np.sum(outlier_mask_1),
    'UV_2_260_mAU': np.sum(outlier_mask_2)
}
return outliers

def detect_peaks(df, noise_dict, window=5):
    peaks_info = {}
    for run in df['run'].unique():
        run_data = df[df['run'] == run].copy()
        run_data = run_data.sort_values('Fraction_number').reset_index(drop=True)

        smoothed_1 = uniform_filter1d(run_data['UV_1_280_mAU'].values, size=window
)
        smoothed_2 = uniform_filter1d(run_data['UV_2_260_mAU'].values, size=window
)

        base_1 = run_data.head(10)['UV_1_280_mAU'].values
        base_2 = run_data.head(10)['UV_2_260_mAU'].values
        sigma_1 = np.std(base_1) if len(base_1) > 1 else 0
        sigma_2 = np.std(base_2) if len(base_2) > 1 else 0

        thresh_1 = noise_dict[run]['UV_1_280_mAU'] + 3 * sigma_1
        thresh_2 = noise_dict[run]['UV_2_260_mAU'] + 3 * sigma_2

        peaks_1, _ = find_peaks(smoothed_1, height=thresh_1, distance=window)
        peaks_2, _ = find_peaks(smoothed_2, height=thresh_2, distance=window)

        peaks_info[run] = {
            'UV_1_280_mAU': {
                'count': len(peaks_1),
                'indices': peaks_1
            },
            'UV_2_260_mAU': {
                'count': len(peaks_2),
                'indices': peaks_2
            }
        }
    return peaks_info

def integrate_peaks(df, noise_dict, peaks_info):

```

```

auc_dict = {}
for run in df['run'].unique():
    run_data = df[df['run'] == run].copy()
    run_data = run_data.sort_values('Fraction_number').reset_index(drop=True)

    smoothed_1 = uniform_filter1d(run_data['UV_1_280_mAU'].values, size=5)
    smoothed_2 = uniform_filter1d(run_data['UV_2_260_mAU'].values, size=5)

    if 'UV_1_280_mAU' in peaks_info[run]:
        peak_indices_1 = peaks_info[run]['UV_1_280_mAU']['indices']
        if len(peak_indices_1) > 0:
            thresh_1 = noise_dict[run]['UV_1_280_mAU'] + 3 * np.std(run_data['UV_1_280_mAU'].values)
            peak_indices_1 = np.where(smoothed_1 > thresh_1)[0]
            auc_1 = np.trapz(smoothed_1[peak_indices_1], run_data['Fraction_number'][peak_indices_1])
        else:
            auc_1 = 0.0
    else:
        auc_1 = 0.0

    if 'UV_2_260_mAU' in peaks_info[run]:
        peak_indices_2 = peaks_info[run]['UV_2_260_mAU']['indices']
        if len(peak_indices_2) > 0:
            thresh_2 = noise_dict[run]['UV_2_260_mAU'] + 3 * np.std(run_data['UV_2_260_mAU'].values)
            peak_indices_2 = np.where(smoothed_2 > thresh_2)[0]
            auc_2 = np.trapz(smoothed_2[peak_indices_2], run_data['Fraction_number'][peak_indices_2])
        else:
            auc_2 = 0.0
    else:
        auc_2 = 0.0

    auc_dict[run] = {
        'UV_1_280_mAU': auc_1,
        'UV_2_260_mAU': auc_2
    }
return auc_dict

def create_visualisations(df, noise_dict, outliers, peaks_info, auc_dict):
    runs = df['run'].unique()

    fig, axes = plt.subplots(2, 2, figsize=(14, 10))

    ax1 = axes[0, 0]
    data_1 = df.groupby('run')['UV_1_280_mAU'].mean().values
    data_2 = df.groupby('run')['UV_2_260_mAU'].mean().values
    ax1.boxplot([data_1, data_2], labels=['UV_1_280_mAU', 'UV_2_260_mAU'])
    ax1.set_title('Boxplots by Run')
    ax1.set_ylabel('Absorbance (mAU)')
    ax1.grid(axis='y', alpha=0.3)

    ax2 = axes[0, 1]
    random_runs = np.random.choice(runs, min(3, len(runs)), replace=False)
    for i, run in enumerate(random_runs):
        run_data = df[df['run'] == run].copy()
        run_data = run_data.sort_values('Fraction_number').reset_index(drop=True)
        ax2.plot(run_data['Fraction_number'], run_data['UV_1_280_mAU'], label=f'

```



```

        Run_{run}', alpha=0.7)
ax2.set_title('Representative_Chromatograms_(UV_1_280_mAU)')
ax2.set_xlabel('Fraction_Number')
ax2.set_ylabel('Absorbance_(mAU)')
ax2.legend()
ax2.grid(True, alpha=0.3)

# Plot peaks and outliers on representative chromatograms
for run in random_runs:
    run_data = df[df['run'] == run].copy()
    run_data = run_data.sort_values('Fraction_number').reset_index(drop=True)
    smoothed_1 = uniform_filter1d(run_data['UV_1_280_mAU'].values, size=5)
    sigma_1 = np.std(run_data['UV_1_280_mAU'].values) if len(run_data) > 1
    else 0
    thresh_1 = noise_dict[run]['UV_1_280_mAU'] + 3 * sigma_1

    # Plot peaks as vertical lines
    peak_indices_1 = np.where(smoothed_1 > thresh_1)[0]
    if len(peak_indices_1) > 0:
        ax2.axvline(x=run_data['Fraction_number'][peak_indices_1], color='red',
                    , linestyle='--', alpha=0.5, label='Detected_Peaks' if run ==
                    random_runs[0] else "")

    # Plot outliers as markers
    vals_1 = run_data['UV_1_280_mAU'].values
    mean_1, std_1 = np.mean(vals_1), np.std(vals_1)
    z_scores_1 = np.abs((vals_1 - mean_1) / std_1)
    outlier_mask_1 = (vals_1 < (np.percentile(vals_1, 25) - 1.5 * (np.
        percentile(vals_1, 75) - np.percentile(vals_1, 25)))) | (z_scores_1 >
        3)
    if np.sum(outlier_mask_1) > 0:
        ax2.scatter(run_data['Fraction_number'][outlier_mask_1], run_data['
            UV_1_280_mAU'][outlier_mask_1], color='orange', s=50, marker='x',
            label='Outliers' if run == random_runs[0] else "")

ax3 = axes[1, 0]
runs_sorted = sorted(runs)
mean_auc_1 = [auc_dict[r]['UV_1_280_mAU'] for r in runs_sorted]
mean_auc_2 = [auc_dict[r]['UV_2_260_mAU'] for r in runs_sorted]
mean_noise_1 = [noise_dict[r]['UV_1_280_mAU'] for r in runs_sorted]
mean_noise_2 = [noise_dict[r]['UV_2_260_mAU'] for r in runs_sorted]

x = np.arange(len(runs_sorted))
width = 0.25

bars1 = ax3.bar(x - width, mean_auc_1, width, label='Mean_AUC_(UV1)')
bars2 = ax3.bar(x, mean_auc_2, width, label='Mean_AUC_(UV2)')
bars3 = ax3.bar(x + width, mean_noise_1, width, label='Mean_Noise_(UV1)')
bars4 = ax3.bar(x + 2*width, mean_noise_2, width, label='Mean_Noise_(UV2)')

ax3.set_title('Mean_AUC_vs_Baseline_Noise_by_Run')
ax3.set_xlabel('Run')
ax3.set_ylabel('Value')
ax3.set_xticks(x + width)
ax3.set_xticklabels(runs_sorted)
ax3.legend()
ax3.grid(axis='y', alpha=0.3)

ax4 = axes[1, 1]

```

```

ax4.axis('off')
table_data = []
for run in runs_sorted:
    table_data.append(f"Run_{run}: UV1_Outliers={outliers[run]['UV_1_280_mAU']}}, UV2_Outliers={outliers[run]['UV_2_260_mAU']}]")
ax4.text(0.1, 0.9, '\n'.join(table_data), transform=ax4.transAxes, fontsize=12,
        verticalalignment='top', fontfamily='monospace', bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))
ax4.set_title('Outlier Counts per Run')

plt.tight_layout()
plt.savefig('/workspace/chromatography_analysis_results.png', dpi=300,
            bbox_inches='tight')
plt.close()

with open('/workspace/outlier_counts.txt', 'w') as f:
    for run in runs_sorted:
        f.write(f"Run_{run}: UV1_Outliers={outliers[run]['UV_1_280_mAU']}}, UV2_Outliers={outliers[run]['UV_2_260_mAU']}]\\n")

def main():
    filepath = '/inputs/chromatography_combined.csv'
    df = load_and_filter_data(filepath)

    noise_dict = calculate_baseline_noise(df)
    outliers = detect_outliers(df, noise_dict)
    peaks_info = detect_peaks(df, noise_dict)
    auc_dict = integrate_peaks(df, noise_dict, peaks_info)

    create_visualisations(df, noise_dict, outliers, peaks_info, auc_dict)
    print("Analysis complete. Visualisations saved to /workspace/")

if __name__ == "__main__":
    main()

```

Listing 2: Code_2

```

import pandas as pd
import numpy as np
from scipy import stats
from scipy.stats import f_oneway, spearmanr

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Debug: Print exact column names and DataFrame shape to verify file content
df_info = {
    'Shape': df.shape,
    'Columns': df.columns.tolist()
}
print("DataFrame Info:", df_info)

# Identify the correct column name dynamically to avoid KeyError
# The column name in the CSV is 'column', but we ensure we select it correctly
# column_name = 'column'

cols_of_interest = ['run', 'column', 'UV_1_280_mAU', 'UV_2_260_mAU']
df_subset = df[cols_of_interest]

```

```

# Convert UV columns to numeric to resolve DtypeWarning and prevent downstream
errors
df_subset['UV_1_280_mAU'] = pd.to_numeric(df_subset['UV_1_280_mAU'], errors='
coerce')
df_subset['UV_2_260_mAU'] = pd.to_numeric(df_subset['UV_2_260_mAU'], errors='
coerce')

# Calculate SNR and Outlier Count per (run, column)
# SNR = Mean / StdDev (using entire column as proxy for noise since baseline is
not explicitly defined)
# Outlier Count: Count values outside IQR * 1.5
snr_data = []

for (run, column), group in df_subset.groupby(['run', 'column']):
    values = group['UV_1_280_mAU']
    mean_val = values.mean()
    std_val = values.std()
    # Handle division by zero: if std is 0, SNR is 0 (no noise, no signal or flat
line)
    if std_val == 0:
        snr = 0.0
    else:
        snr = mean_val / std_val

    # Outlier Count using IQR
    Q1 = values.quantile(0.25)
    Q3 = values.quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outlier_count = ((values < lower_bound) | (values > upper_bound)).sum()

    snr_data.append({'run': run, 'column': column, 'SNR': snr, 'Outlier_Count':
outlier_count})

snr_df = pd.DataFrame(snr_data)

# Debug: Check if 'column' column exists in snr_df
print(f"Columns in snr_df: {snr_df.columns.tolist()}")

# Aggregate to Run level
# Calculate Avg SNR and Total Outlier Count per run
run_agg = snr_df.groupby('run').agg({
    'SNR': 'mean',
    'Outlier_Count': 'sum'
}).reset_index()
run_agg.columns = ['run', 'Avg-SN-R', 'Total_Outlier_Count']

# Re-add the 'column' column to run_agg as it was dropped during groupby
# Check if 'column' exists in snr_df first
if 'column' in snr_df.columns:
    # Use transform to carry forward the first 'column' value for each run
    run_agg['column'] = snr_df.groupby('run')['column'].transform('first')
else:
    print(f"WARNING: 'column' column not found in snr_df. Columns available: {
snr_df.columns.tolist()}")

# Spearman Correlation

```

```

# Convert 'column' to numeric if possible, else rank it
run_agg['column_numeric'] = pd.to_numeric(run_agg['column'], errors='coerce')
if run_agg['column_numeric'].isna().any():
    unique_cols = run_agg['column'].unique()
    col_rank = pd.Series(range(1, len(unique_cols)+1), index=unique_cols)
    run_agg['column_numeric'] = run_agg['column'].map(col_rank)

corr_sn, p_val_sn = spearmanr(run_agg['column_numeric'], run_agg['Avg_SNR_R'])
corr_out, p_val_out = spearmanr(run_agg['column_numeric'], run_agg['
    Total_Outlier_Count'])

# ANOVA: Test for significant differences in mean peak height/area across run
# groups
# Use raw data grouped by run
groups = [group['UV_1_280_mAU'].values for _, group in df_subset.groupby('run')]
groups = [g for g in groups if len(g) > 0]

if len(groups) >= 2:
    f_stat, p_val_anova = f_oneway(*groups)
else:
    f_stat = 0
    p_val_anova = 1

# Identify top 3 runs requiring action
# Sort by Total_Outlier_Count descending, then Avg_SNR_R ascending
run_agg_sorted = run_agg.sort_values(by=['Total_Outlier_Count', 'Avg_SNR_R'],
    ascending=[False, True]).head(3)

# Prepare output
output_lines = []
output_lines.append(f"Spearman_Correlation_(Column_vs_SNR):_r={corr_sn:.4f},_p={
    p_val_sn:.4f}")
output_lines.append(f"Spearman_Correlation_(Column_vs_Outlier_Count):_r={corr_out
    :.4f},_p={p_val_out:.4f}")
output_lines.append(f"ANOVA_F-statistic:_{f_stat:.4f},_p-value:_{p_val_anova:.4f}"
    )
output_lines.append("")
output_lines.append("Top_3_Runs:")
output_lines.append("Run_ID_|_Outlier_Count_|_Avg_SNR_|_Flag")
for _, row in run_agg_sorted.iterrows():
    # Flag logic: If Outlier Count > 2, 'Potential Failure'. Else 'High
    Variability'.
    flag = 'Potential_Failure' if row['Total_Outlier_Count'] > 2 else 'High_
    Variability'
    output_lines.append(f"{row['run']}|_{row['Total_Outlier_Count']}|_{row['
    Avg_SNR_R']:.2f}|_{flag}")

# Print output
print('\n'.join(output_lines))

# Save to /workspace/
output_text = '\n'.join(output_lines)
with open('/workspace/analysis_step3_output.txt', 'w') as f:
    f.write(output_text)

```

Listing 3: Code_3